

1 Introduction

commutative한 프로그래밍 언어를 만들기 위해서 많은 시도를 했다. 처음에는 Haskell의 Hask 범주와 같은 범주론 기반 언어의 범주를 적절한 (full) functor를 통해 Abelian 범주로 변환하면 함수 합성 단위에서 commutative한 언어가 될 것이라고 생각했다. 물론 그게 가능하다면 그렇게 되었겠지만, Cartesian Closed Category는 Abelian이 될 수 없다고 한다. 그 다음은 1차 논리식의 논리 연산자 단위에서 commutative한 것이 가능한지 보았다. 그건 심지어 assosiative 하지도 않았기 때문에 다른 방법을 찾아봤다. 모든 (부분)재귀함수를 나타내기 위한 좋은 formal system이 있을까? 자연스러운 방법은 찾을 수 없었다. 그렇기 때문에 언어의 사용자들이 직접 지정한 generator의 free commutative monoid에서 (부분)재귀함수 전체의 집합으로 가는 함수를 임의로 정의할 수 있도록 하였다. **SCP- n** (Standard Commutative Processor n)은 commutative한 언어의 정의를 누구나 쉽게 하고 싶은 대로 할 수 있도록 준비된 도구이며 번역기의 역할을 한다.

2 SCP- n

따로 적지 않은 용어는 일반적으로 쓰이는 대로 정의한다.

Definition 2.1. A *commutative language* $L \subset \Sigma^*$ is a subset of the free commutative monoid Σ^* generated by given nonempty set Σ .

SCP- n 은 재귀적으로 정의되며 3가지 부위로 구성된다.

1. **definition 2.1**의 set Σ 의 모든 원소를 한 번씩 쓴 문자열.

Σ^* 를 정의하기 위해서 generating set Σ 의 모든 원소를 적어 줄 필요가 있다. 언어에 사용할 알파벳을 고르는 일이라고 생각하면 된다. 예시로 만든 SCP-1에서는 유니코드 문자 전체의 집합의 power set에서 Σ 를 골라야 한다.

2. 적절한 (부분)재귀함수 $\varphi : \Sigma^* \rightarrow \mathbb{N}$ 을 나타내는 **SCP- $(n-1)$** 코드.

SCP- $(n-1)$ 인터프리터 : **SCP- $(n-1)$** $\rightarrow$ (부분)재귀함수 전체의 집합을 i 라 하자.

Σ^* 는 $\mathbb{N}$ 유한 개의 direct sum과 isomorphic하기 때문에 φ 를 numeric function으로 간주하고 부분재귀함수로 정의할 수 있다.

1에서 정의한 Σ^* 전체에서 자연수 집합 $\mathbb{N}$ 으로 가는 함수를 직접 정의한다. φ 는 **SCP- n** 인터프리터에 포함된 함수 $\psi : \mathbf{SCP-}(n-1) \rightarrow \mathbb{N}$ 의 역함수와 가능할 경우 합성되어 i 를 통해 $L \subset \Sigma^*$ 의 일부 원소를 (부분)재귀함수들에 대응시킨다. $\Sigma^* \setminus L$ 의 원소는 φ 를 통해 $\mathbb{N} \setminus \text{im } \psi$ 로 보내지고 값이 정의되지 않는다.

3. L 의 문자열 s .

Σ^* 의 연산은 juxtaposition이다. permutation들은 모두 동일한 원소로 취급된다.

1과 2는 언어를 정의하는 부분이고, 실제 프로그램은 s 하나이다. s 는 1, 2에서 정의된 언어의 원소로 간주되어 $\psi^{-1}\varphi(s) \in \mathbf{SCP-}(n-1)$ 로 변환된다. i 를

거칠 경우 (부분)재귀함수로 변환된다. (마지막 과정은 **SCP- n** 인터프리터에 포함되어 있지 않다. 언어 간의 변환만 다루기 때문에 실제로 $\psi^{-1}\varphi(s)$ 가 **SCP- $(n-1)$** 에서 무슨 의미를 갖는지는 생각하지 않는다. 물론 i 는 $\psi^{-1}\varphi(s)$ 가 무슨 함수를 나타내는지 알고 있겠지만, 우리는 언어나 다른 표현을 거치지 않고서 함수를 직접 인식할 수 없기 때문에 마지막에는 언어로 표현되도록 한 것이다. 물론 함수를 실행시키는 것은 가능하지만 그 경우 입력값을 같이 넣어 줘야 하며 numeral의 표현도 쉬운 일이 아니므로 언어가 훨씬 복잡해질 것이다.)

1과 2는 fix할 수 있다. 모든 (부분)재귀함수를 나타내기 위해서는 2에서 적절한 함수를 골라야 한다. 어떤 함수는 더 적절(standard)한데, 사용자가 적은 함수가 적절한지 아닌지 (재귀적으로) 알 수 있는 방법은 없기 때문에 그런 건 검사하지 않는다.

예시로 만든 **SCP-1**의 경우, i 는 소수를 이용한 인코딩이다. 지금은 문자 단위로 되어 있는데 함수 단위로 번호를 매기는 것이 나올지도 모른다.

3 SCP-0

SCP- n 은 **SCP-0**부터 위에서 말한 방법대로 재귀적으로 정의된다. 임의의 언어를 **SCP-0**로 선택할 수 있지만, 전체 프로그램을 1줄로 간편하게 적을 수 있도록 언어를 새로 만들었다. 인간도 쉽게 읽을 수 있는 간단한 언어이다. 여기서 제공된 **SCP-0** 인터프리터는 **SCP-0** 문장을 $\mathbb{N} \times \dots \times \mathbb{N} \rightarrow \mathbb{N}$ 부분재귀함수로 변환한다. 동봉된 `scpi`는 함수에 함숫값을 넣어 계산한다.

4 Outroduction

아무 언어나 골라서 무한히 반복할 수 있다. 이 경우 코드를 다시 짜고 컴파일을 다시 해야 하기 때문에 아직은 코드 작성 언어인 Haskell에 의존하고 있지만 조금 더 완성하면 완전히 독립적인 무한 반복기가 될 것이다.

github